

Plano de Reintegração Técnica: Ravena AI - Sovereign Elite Edition

Autor: Manus AI

Data: 20 de Abril de 2026

Versão: 1.0

1. Introdução

Este documento detalha o **Plano de Reintegração Técnica** para a Ravena AI - Sovereign Elite Edition. O objetivo é reativar funcionalidades e lógicas operacionais cruciais que foram identificadas como ausentes ou simplificadas nos relatórios de alto nível, mas que são fundamentais para a performance, segurança e autonomia do sistema. Este plano serve como um guia passo a passo para integrar as especificações recuperadas no arquivo `config_v3_reintegrated.json` ao código-fonte e configurações existentes da Ravena AI.

Arquivo de Configuração Mestre: `/home/ubuntu/config_v3_reintegrated.json`

2. Visão Geral do Plano

O plano está dividido em três áreas principais de reintegração:

- Configuração Dinâmica da SignalBridge:** Reativar o Gatilho de Suitability Dinâmico.
- Estabilidade da Interface Telegram:** Reintegrar os parâmetros de geração e interceptores de saudações.
- Autonomia do Núcleo Omega:** Reativar a lógica de autocorreção e fusão visual-semântica.

Cada seção abaixo fornecerá instruções detalhadas sobre quais arquivos modificar e como aplicar as alterações.

3. Reintegração Detalhada dos Módulos

3.1. Reintegração do Gatilho de Suitability Dinâmico (SignalBridge)

Esta etapa visa reativar a lógica de ajuste de risco da `SignalBridge` baseada no saldo em USDT, conforme detalhado no `relatorio_v220.md` 1.

Arquivos a Modificar:

- `src/signal_bridge.py`
- `config_v3.json` (ou o arquivo de configuração principal da Ravena)

Instruções:

1. Atualizar `config_v3.json` :

- Abra o arquivo `config_v3.json` principal da Ravena.
- Copie e cole a seção `suitability_dynamic_gate` do `config_v3_reintegrated.json` para o seu `config_v3.json`.
- Exemplo de estrutura a ser adicionada:

JSON

```
"suitability_dynamic_gate": {
  "description": "Recuperado da v2.2.0: Ajuste automático de risco baseado em",
  "modes": {
    "AGGRESSIVE": {
      "balance_limit": 10000,
      "sentiment_threshold": 0.15,
      "audit_required": false,
      "visual_confirmation": false
    },
    "MODERATE": {
      "balance_limit": 50000,
      "sentiment_threshold": 0.25,
      "audit_required": true,
      "visual_confirmation": true
    },
    "CONSERVATIVE": {
      "balance_limit": 999999,
      "sentiment_threshold": 0.35,
      "audit_required": true,
      "visual_confirmation": true,
      "probability_bonus": 0.05
    }
  }
},
```

2. Modificar `src/signal_bridge.py` :

- Abra `src/signal_bridge.py`.
- Localize a função ou classe responsável por determinar o perfil de risco ou suitability.
- Implemente a lógica para ler a configuração `suitability_dynamic_gate` do `config_v3.json`.

- Crie uma função `determine_suitability_mode(current_balance: float) -> str` que, com base no `current_balance` e nos `balance_limit` definidos na configuração, retorne o modo (`AGGRESSIVE` , `MODERATE` , `CONSERVATIVE`).
- Integre este modo na lógica de processamento de sinais, aplicando as `sentiment_threshold` , `audit_required` , `visual_confirmation` e `probability_bonus` correspondentes.

3.2. Reintegração da Estabilidade da Interface Telegram

Esta etapa visa restaurar a estabilidade e a naturalidade das interações do bot Telegram, conforme detalhado no `telegram_v19.md` 3 .

Arquivos a Modificar:

- `src/telegram_bot.py` (ou o arquivo que gerencia a interação com o Telegram)
- `src/ravena_model.py` (ou o arquivo que gerencia a geração de texto do modelo de IA)
- `config_v3.json`

Instruções:

1. Atualizar `config_v3.json` :

- Copie e cole a seção `telegram_v19_stability` do `config_v3_reintegrated.json` para o seu `config_v3.json` .

2. Modificar `src/telegram_bot.py` :

- Implemente a lógica de `interceptors.greetings` . Antes de enviar a mensagem do usuário para o modelo de IA, verifique se ela corresponde a uma das saudações configuradas. Se sim, responda com `default_greeting` e não acione o modelo de IA.
- Certifique-se de que o modelo de linguagem `pierreguillou/gpt2-small-portuguese` seja carregado, se ainda não for o caso.

3. Modificar `src/ravena_model.py` :

- Ajuste os parâmetros de geração do modelo de IA (`temperature` , `top_k` , `top_p` , `repetition_penalty` , `no_repeat_ngram_size`) para os valores especificados em `telegram_v19_stability.generation_params` no `config_v3.json` .

3.3. Reintegração da Autonomia do Núcleo Omega

Esta etapa visa reativar as capacidades de autocorreção e a fusão visual-semântica do Núcleo Omega, conforme detalhado no `autonomia_ravena.md` 2 .

Arquivos a Modificar:

- `src/omega.py` (ou o arquivo do núcleo orquestrador)

- `src/vision_rag_semantic.py` (se o arquivo não existir, será necessário criá-lo)
- `config_v3.json`

Instruções:

1. Atualizar `config_v3.json` :

- Copie e cole a seção `omega_autonomous_reintegration` do `config_v3_reintegrated.json` para o seu `config_v3.json` .

2. Criar/Modificar `src/vision_rag_semantic.py` :

- Se o arquivo não existir, crie-o. Ele deve conter a lógica para decodificar `SnapshotVisual` em `DecisaoAutonoma` consultando o RAG e mapeando anomalias para ações, conforme descrito na seção `vision_semantic_fusion.anomaly_mapping` do `config_v3.json` .

3. Modificar `src/omega.py` :

- Implemente a função `processar_percepcao_visual` que recebe um `SnapshotVisual` e delega a decodificação ao `VisionRAGSemantic` .
- Integre a lógica de autocorreção: se `self_healing.active` for `true` e a confiança da `DecisaoAutonoma` for maior que `self_healing.confidence_threshold` , inicie os procedimentos de autocorreção (ex: `auto_restart_services`). Monitore `max_auto_correction_cycles` .
- Certifique-se de que o `omega.py` utilize o `auditor_v3.py` para validação de ferramentas, conforme a política `security_auditor_policy` no `config_v3.json` .

3.4. Reintegração da Política de Auditoria de Segurança (Auditor v3.0)

Esta etapa garante que a política de segurança de ferramentas de terceiros seja aplicada consistentemente ² .

Arquivos a Modificar:

- `src/auditor.py` (ou `auditor_v3.py`)
- `config_v3.json`

Instruções:

1. Atualizar `config_v3.json` :

- Copie e cole a seção `security_auditor_policy` do `config_v3_reintegrated.json` para o seu `config_v3.json` .

2. Modificar `src/auditor.py` :

- Certifique-se de que o `auditor.py` (ou `auditor_v3.py`) leia e aplique as configurações de `analysis_mode` , `sandbox_execution` , `forbidden_patterns` e `network_audit` do

`config_v3.json` .

- Verifique se a análise AST está ativa e buscando os `forbidden_patterns` .
-

4. Testes e Validação

Após a implementação de cada etapa, é crucial realizar testes rigorosos:

- **Testes Unitários:** Verifique se os testes unitários existentes para `SignalBridge` , `TelegramBot` , `Omega` e `Auditor` continuam passando.
 - **Testes de Integração:** Crie cenários de teste que validem as novas lógicas:
 - Simule diferentes saldos para verificar a mudança do modo de suitability.
 - Envie saudações ao bot Telegram e verifique a resposta padrão.
 - Simule anomalias visuais para testar a autocorreção do Omega.
 - Tente integrar uma ferramenta com padrões proibidos para verificar a atuação do Auditor.
 - **Monitoramento:** Monitore o comportamento do sistema em ambiente de teste para garantir que as mudanças não introduziram efeitos colaterais indesejados.
-

5. Conclusão

Este Plano de Reintegração Técnica fornece um roteiro claro para restaurar a plenitude da inteligência e resiliência da Ravena AI. Ao seguir estas instruções, você garantirá que a Sovereign Elite Edition opere com todas as capacidades projetadas, aproveitando o conhecimento técnico detalhado que havia sido temporariamente obscurecido. A Ravena AI estará, assim, verdadeiramente completa e blindada.

Referências

- [1] Relatório Técnico: Ponte de Dados — Tradutor de Sinais v2.2.0 (1RgLHuAGMw14MgfBaPD_fy13zLGnwJv4h).
- [2] Relatório de Implementação e Validação: Orquestração Autônoma e Percepção Avançada (17zX1FNZe6ftQ75QA-mWTiysznZmtuXHp).
- [3] Relatório de Consolidação: Ravena AI - Sistema Cognitivo Modular V1.9-TELEGRAM-READY (17ybhELhNuui-MSq3xtjL558qYoga35Hy).
- [4] Auditor de Ferramentas de Terceiros (auditor_v3.py).